

Шпаргалка по Git

Команды Git, которые нужны в повседневной работе: настройка, ветки, история, обмен изменениями с удалённым репозиторием и откат ошибок. Квадратные скобки — место для вашего значения, сами скобки набирать не нужно.

Настройка

Задаётся один раз для всех репозиториев пользователя.

```
git config --global user.name "[имя]"
    Имя, которое подставляется в ваши коммиты

git config --global user.email "[почта]"
    Почта автора коммитов

git config --global init.defaultBranch main
    Имя ветки по умолчанию в новых репозиториях

git config --list
    Показать текущие настройки и откуда они взяты
```

Создание репозитория

```
git init
    Превратить текущий каталог в репозиторий Git

git clone [url]
    Скачать существующий репозиторий вместе со всей
    историей и ветками

git clone [url] [каталог]
    То же, но в каталог с выбранным именем

git remote add origin [url]
    Привязать локальный репозиторий к удалённому
```

Повседневный цикл

```
git status
    Что изменено, что подготовлено к коммиту, на какой
    вы ветке

git add [файл]
    Подготовить изменения файла к коммиту

git add -p
    Выбрать изменения для коммита по частям, кусок за
    куском

git commit -m "[сообщение]"
    Зафиксировать подготовленные изменения в истории

git commit --amend
    Переписать последний коммит: сообщение или состав
    файлов

git rm [файл]
    Удалить файл и зафиксировать удаление

git mv [откуда] [куда]
    Переместить или переименовать файл с сохранением
    истории
```

Ветки

Ветка — это подвижный указатель на коммит. Коммит всегда попадает в ту ветку, на которой вы находитесь.

```
git branch
    Список локальных веток, текущая отмечена
    звёздочкой

git switch -c [ветка]
    Создать ветку и сразу перейти на неё

git switch [ветка]
    Перейти на существующую ветку

git merge [ветка]
    Влить историю указанной ветки в текущую

git rebase [ветка]
    Перенести свои коммиты поверх указанной ветки —
    история остаётся линейной

git branch -d [ветка]
    Удалить ветку, изменения которой уже влиты
Старый вариант: git checkout [ветка] и git checkout
    -b [ветка] делают то же, что switch.
```

Обмен с удалённым репозиторием

```
git fetch
    Скачать новые коммиты, ничего не меняя в рабочем
    каталоге

git pull
    Скачать и сразу влить изменения в текущую ветку
    (fetch + merge)

git pull --rebase
    То же, но свои коммиты переносятся поверх чужих

git push
    Отправить коммиты текущей ветки в удалённый
    репозиторий

git push -u origin [ветка]
    Отправить новую ветку и связать её с удалённой

git remote -v
    Показать адреса привязанных удалённых
    репозиториев
```

История и различия

```
git log --oneline --graph
    Компактная история со схемой ветвлений

git log --follow [файл]
    История файла, включая переименования

git diff
    Изменения, ещё не подготовленные к коммиту

git diff --staged
    Изменения, подготовленные к коммиту

git diff [ветка1]..[ветка2]
    Разница между двумя ветками

git show [коммит]
    Содержимое и описание конкретного коммита

git blame [файл]
    Кто и в каком коммите менял каждую строку
```

Отмена изменений

`git restore [файл]`
Вернуть файл к состоянию последнего коммита

`git restore --staged [файл]`
Убрать файл из подготовленных, изменения сохранить

`git reset [коммит]`
Отменить коммиты после указанного, изменения оставить в рабочем каталоге

`git reset --hard [коммит]`
Вернуться к указанному коммиту, отбросив все изменения

`git revert [коммит]`
Создать новый коммит, отменяющий указанный — история не переписывается

`git reflog`
Журнал перемещений HEAD: помогает вернуть «потерянные» коммиты

Осторожно: `reset --hard`, `rebase` и `commit --amend` переписывают историю. Если коммиты уже отправлены в общий репозиторий, у коллег сломается их копия — в таких случаях предпочтителен `revert`.

Отложить работу (stash)

`git stash`
Спрятать незавершённые изменения и вернуть чистый рабочий каталог

`git stash list`
Список отложенных наборов изменений

`git stash pop`
Вернуть последний отложенный набор и убрать его из списка

`git stash drop`
Удалить отложенный набор, не применяя

Теги

`git tag [имя]`
Пометить текущий коммит, например номером версии

`git tag -a [имя] -m "[описание]"`
Аннотированный тег с автором и описанием

`git push origin [имя]`
Отправить тег в удалённый репозиторий

Файл .gitignore

Перечислите в нём то, что не должно попадать в репозиторий: сборки, зависимости, локальные настройки, файлы с секретами. Шаблоны для языков и фреймворков — на github.com/github/gitignore.

`git check-ignore -v [файл]`
Показать, какое правило игнорирует файл

`git rm --cached [файл]`
Перестать отслеживать файл, оставив его на диске

Словарь

- **Репозиторий** — проект вместе со всей историей изменений.
- **Коммит** — снимок состояния проекта с автором, датой и описанием.
- **Ветка** — подвижный указатель на коммит; параллельная линия работы.
- **HEAD** — указатель на текущее положение: обычно вершина активной ветки.
- **Индекс (staging area)** — промежуточная область, куда попадают изменения перед коммитом.
- **origin** — принятое имя основного удалённого репозитория.
- **Форк** — ваша копия чужого репозитория на сервере.
- **Pull request** — предложение влить изменения ветки, с обсуждением и проверками.
- **Конфликт слияния** — ситуация, когда один участок изменён по-разному в двух ветках; Git просит выбрать ручную.